Firecrab 발표 대사

2026년 오픈소스 개발자대회 / JustForFun / 박용현 / 접수번호 3633

본편 14장, 90초 시연 포함. 목표 시간은 10분이며 리허설에서 조정한다.

회색 괄호는 화면 조작 지시다. 발표할 때 읽지 않는다.

1페이지 Firecrab

목표 10초 · 누적 00:10

안녕하세요. 멀티 AI 에이전트 구축과 서버리스 배포·운영을 위한 플랫폼, Firecrab을 소개하겠습니다.

2페이지 팀 소개

목표 20초 · 누적 00:30

JustForFun의 박용현입니다. 프로젝트의 기획과 설계, 개발과 검증을 담당했습니다. AI 작업 환경과 서비스 운영을 함께 관리하는 데 초점을 맞췄습니다.

3페이지 프로젝트 개요

목표 25초 · 누적 00:55

Firecrab은 AI 작업부터 앱 배포와 운영까지 한곳에서 관리하는 오픈소스 플랫폼입니다.

Sandbox와 Canvas에서 AI 작업과 결과를 다루고, MicroVM과 Cluster에서 실행 환경을 관리합니다. Deploy로 앱을 배포한 뒤에는 로그와 상태를 확인합니다.

이 기능들을 사용자 서버와 클러스터에서 운영합니다. 이런 구성이 필요한 배경부터 보겠습니다.

4페이지 개발 배경과 목적

목표 45초 · 누적 01:40

Firecrab은 처음부터 AI 플랫폼으로 시작한 프로젝트는 아닙니다.

처음에는 오픈소스 Firecracker를 관리할 인프라 대시보드를 만들었습니다. 직접 운영하면서 VM과 네트워크, 디스크에 필요한 기능을 순서대로 구현했습니다.

그다음에는 앱 배포와 업데이트, 로그 확인 등 운영에 필요한 도구를 붙였습니다.

이 실행 환경을 AI 작업에도 활용하기 위해 AI Sandbox와 협업 AI 작업 공간인 Canvas로 범위를 넓혔습니다.

직접 운영하면서 필요한 기능을 더한 결과가 지금의 통합 구조입니다.

5페이지 플랫폼 비교

목표 45초 · 누적 02:25

Replit은 AI로 앱을 만들고 배포하는 플랫폼입니다. Modal은 함수와 AI 연산, Sandbox 실행 환경을 제공합니다. Vercel은 웹 배포와 함수, Sandbox를 제공합니다. Fly.io는 VM과 앱 운영을 다루고, Coolify는 사용자 서버에 앱과 DB를 배포합니다.

Firecrab의 차이는 이 기능들을 어디서 어떻게 함께 관리하는지에 있습니다.

관리형 플랫폼과 비교하면 보유 서버에 직접 설치하고 자원과 실행 위치를 선택합니다. 배포 도구와 비교하면 앱 배포에 AI 작업과 변경 검토, MicroVM 실행 환경을 결합합니다.

AI Sandbox와 Canvas, 배포와 로그 확인을 같은 콘솔에서 다룹니다.

6페이지 올인원 통합 구조

목표 30초 · 누적 02:55

Firecrab의 구조는 세 영역으로 나눌 수 있습니다.

AI 기능에는 작업을 실행하고 검토하는 Sandbox와 협업 공간인 Canvas가 있습니다. 실행 인프라에는 MicroVM과 여러 호스트를 관리하는 Cluster가 있습니다. 서비스 운영에는 배포와 업데이트, 상태 확인과 복구 기능이 있습니다.

작업 결과와 실행 환경, 배포 상태를 같은 콘솔에서 확인하고 관리합니다. AI Canvas도 이 안의 AI 기능 중 하나입니다.

먼저 AI Sandbox부터 보겠습니다.

7페이지 AI Sandbox

목표 50초 · 누적 03:45

AI Sandbox는 AI가 만든 결과를 실제 실행 결과와 함께 검토하는 기능입니다.

프로젝트와 모델을 선택하면 프로젝트 사본에서 작업을 시작합니다. 명령과 테스트는 선택한 MicroVM에서 실행합니다. MicroVM은 작업 환경을 분리하는 가벼운 가상머신입니다.

사용자는 어떤 파일이 바뀌었는지, 어떤 명령을 실행했는지, 테스트 결과가 무엇인지 확인할 수 있습니다. 검토한 뒤에는 선택한 파일을 원본 프로젝트에 반영합니다.

대화에서 받은 코드와 실행·검토 결과를 같은 작업 기록에서 확인하는 구성입니다.

이제 이런 작업을 여러 에이전트로 나누는 방식을 보겠습니다.

8페이지 멀티 AI 에이전트

목표 40초 · 누적 04:25

작업이 커지면 에이전트마다 역할을 나눌 필요가 있습니다.

예를 들어 로그인 기능을 만든다면, API를 구현하는 작업과 화면을 구현하는 작업을 나눌 수 있습니다. 선행 작업의 결과와 변경 파일은 다음 작업의 사본에 전달합니다.

사용자는 각 작업의 명령과 테스트, 산출물을 확인할 수 있습니다. 무엇을 실행했고 어떤 변경을 반영했는지도 기록으로 확인합니다.

현재 프로젝트 내 실행은 순서대로 진행하며, 재시도는 명시적으로 요청합니다.

이 작업들을 함께 살펴보는 공간이 AI Canvas입니다.

9페이지 AI Canvas

목표 40초 · 누적 05:05

AI Canvas는 사람과 AI 에이전트가 함께 쓰는 협업 AI 작업 공간입니다.

지금 보이는 화면은 테스트 데이터로 만든 UI 목업입니다.

왼쪽에는 프로젝트 자료와 작업 목록이 있습니다. 캔버스에는 역할별 에이전트와 터미널, 공유 파일과 문서가 배치되어 있습니다. 앱 미리보기와 변경 기록도 같은 공간에서 확인하는 구성입니다.

사용자가 대화와 파일, 실행 결과를 따로 찾아다니지 않고 작업 맥락을 함께 확인하도록 구성했습니다.

Canvas는 Firecrab의 AI 기능 중 하나입니다. 실제 코드 실행은 Sandbox의 격리 환경에서 다룹니다.

10페이지 AI 실행 보안

목표 50초 · 누적 05:55

AI가 만든 코드를 실행할 때는 어디서 실행하고 어디까지 접근할 수 있는지가 중요합니다.

Firecrab은 별도의 운영체제 커널을 가진 MicroVM에서 작업을 실행합니다. 전용 사용자와 네트워크 규칙으로 실행 권한과 연결 범위를 설정합니다.

모델 연결에 사용하는 자격증명은 API 호스트에서 관리하고, 작업용 가상머신과 모델에 전달하는 내용에서 분리합니다.

원본 코드 반영도 별도 단계입니다. 사용자가 변경 내용을 검토하고 적용할 파일을 선택합니다.

격리 실행과 결과 검토를 거쳐 서비스 배포로 이어지는 구성을 보겠습니다.

11페이지 서버리스 배포·운영

목표 50초 · 누적 06:45

앱 배포는 준비한 OCI 이미지를 기준으로 진행합니다. OCI 이미지는 앱과 실행에 필요한 파일을 묶은 표준 이미지입니다.

AI 작업 결과를 검토해 코드를 반영하고, 이미지를 준비한 뒤 Deploy에서 배포합니다. 코드 반영과 이미지 준비, 배포는 각각 확인하는 단계입니다.

배포할 때는 HTTP 준비 상태를 확인합니다. 이후에는 업데이트와 롤백을 관리합니다.

이미지 저장소나 DB 같은 외부 서비스의 연결 정보는 PIPE에서 관리합니다.

이제 시연에서 AI 작업 결과 검토와 MicroVM 확인, 앱 배포와 로그 확인을 보여 드리겠습니다.

12페이지 기능 시연

목표 90초 · 누적 08:15

(시연 전에 AI 작업 결과와 앱 이미지를 준비한다. 실제 화면을 확인한 뒤 다음 대사를 읽는다.)

00-20초 AI 작업 결과 검토

(AI Sandbox의 작업 결과를 연다.)

먼저 준비한 AI 작업 결과입니다. 변경 파일과 실행한 명령, 테스트 결과를 확인합니다. 이 내용을 검토하고 원본에 반영할 파일을 선택합니다.

20-40초 MicroVM 실행 환경 확인

(작업에 연결된 MicroVM의 상태와 콘솔을 보여 준다.)

다음은 이 작업에 연결된 MicroVM입니다. 실행 환경과 할당 자원, 콘솔을 확인할 수 있습니다.

40-70초 이미지 배포

(미리 준비한 이미지를 선택하고 Deploy 실행과 준비 상태를 확인한다.)

검토한 코드로 미리 준비한 이미지를 배포하겠습니다. 배포 요청 뒤에는 앱의 준비 상태와 실행 상태를 확인합니다.

70-90초 앱과 로그 확인

(앱에 접속하고 배포 상태·로그·이벤트를 보여 준다.)

마지막으로 앱에 접속하고 로그와 이벤트를 확인합니다. AI 작업 결과 검토부터 실행 환경 확인, 앱 배포와 운영 상태 확인까지의 구성입니다.

13페이지 활용 방안과 계획

목표 80초 · 누적 09:35

주요 사용자는 세 가지로 구분했습니다.

첫째는 보유 서버를 활용하려는 사용자입니다. 서버 자원으로 앱과 AI 작업을 운영하고, 추가 관리형 플랫폼 요금을 줄일 수 있습니다. 서버와 전력, 모델 API 비용은 별도입니다.

둘째는 테스트 사용자입니다. AI가 만든 코드를 격리 환경에서 실행하고 변경 결과와 테스트를 검토합니다.

셋째는 간단한 웹앱을 운영하는 개발자입니다. 작은 앱을 배포하고 상태와 로그를 확인합니다.

개발 마일스톤은 네 가지입니다. AI Canvas의 협업 화면과 작업 결과 연계를 고도화합니다. AI 보안은 권한과 네트워크 제한, 복구 검증을 확대합니다. 외부 플랫폼 연동은 DB와 Storage, Secret과 개발 도구 연결을 넓힙니다.

장기적으로는 설치 문서와 기여 안내를 보완하고 오픈소스 기여자를 확보하겠습니다.

코드와 문서는 Apache-2.0으로 공개합니다. Issue와 PR로 개선에 참여할 수 있습니다.

14페이지 마무리

목표 25초 · 누적 10:00

Firecrab은 AI 작업과 서비스 운영을 한곳에서 관리하는 플랫폼입니다.

AI 협업과 MicroVM 격리, 배포·운영을 보유 서버와 클러스터에서 함께 다룹니다.

Firecracker 관리 도구에서 시작해 직접 운영하는 AI 개발 환경으로 확장했습니다.

이상으로 발표를 마칩니다.

부록 대사

본 발표 시간에는 포함하지 않는다. 관련 질문에 사용한다.

15페이지 Q&A

질문에 답한다. 필요한 경우 크로스 플랫폼 구조와 사용 오픈소스 부록을 보여 준다.

16페이지 크로스 플랫폼

운영체제별로 실행 환경을 준비하고 공통 Linux 계층에서 Firecrab을 실행합니다.

Linux는 systemd와 KVM을 직접 사용합니다. macOS는 Apple의 Virtualization.framework로 Debian 관리 VM을 실행합니다. Windows는 WSL2의 Debian 관리 환경을 사용합니다.

그 위에서 같은 Firecrab API와 웹 콘솔, Firecracker 기반 MicroVM을 사용합니다.

macOS는 중첩 가상화 지원이 필요하고 M5 실행 기록이 있습니다. Windows는 Preview 범위입니다. 물리 PC와 ARM64는 추가 검증 범위를 구분합니다.

17페이지 사용 오픈소스

실제 사용하는 오픈소스와 역할입니다.

Firecracker는 MicroVM을 실행합니다. Tokio와 Axum은 비동기 작업과 HTTP API를 처리합니다. 웹 콘솔은 React, 브라우저 터미널은 Ghostty Web입니다. rustls는 TLS 암호화 통신을 처리합니다.

AI API 연동에는 reqwest와 Serde를 사용합니다. reqwest는 모델 API 요청을 보내고, Serde는 도구 호출과 응답의 JSON을 처리합니다.

AI 작업 제어는 직접 구현합니다. OpenAI와 Anthropic 모델은 외부 API로 연동합니다.